End to End AI Agent Chatbot with FastAPI, LangGraph, Langchain

Phase 1: Setup AI Agent

1. Project Environment Setup

You set up a Python development environment using **Python 3.11** and created a **virtual environment** for the project. All necessary libraries for LLMs and tools were installed via Pipenv, including packages for LangChain, LangGraph, and environment variable management. This ensures the project runs in an isolated and consistent environment.

2. API Key Management

You created a .env file to securely store API keys for:

- **Groq**
- **OpenAI**
- **Tavily**

These keys were loaded into the project using environment variable management, allowing the agent to authenticate with each external service without hardcoding sensitive information.

3. Language Model Integration

You integrated two language models:

- **Groq LLaMA model** for advanced reasoning and tool integration.
- **OpenAI GPT-4o-mini model** for additional LLM support.

This allows the agent to handle general AI tasks and leverage multiple models for more robust responses.

4. Tool Integration

You integrated the **Tavily search tool**, which enables the agent to fetch real-time information from the web. This makes the chatbot capable of answering queries that require up-to-date knowledge, such as trends in the crypto market.

5. Agent Creation

You created a **ReAct-style agent** that combines the Groq model with the T

avily search tool. The agent is designed to:

- Understand user queries
- Decide when to use tools
- Produce a final response that incorporates retrieved information

6. Message Handling

You implemented a **message state system**, maintaining a conversation flow between the system, the user, and the AI. This allows the agent to process multiple messages, maintain context, and generate meaningful responses.

7. Agent Invocation

You successfully ran the agent, passing user queries through the system. The agent called the appropriate tools when necessary and generated a structured response. You also implemented a way to extract the AI's final output from the returned response messages.

8. Results

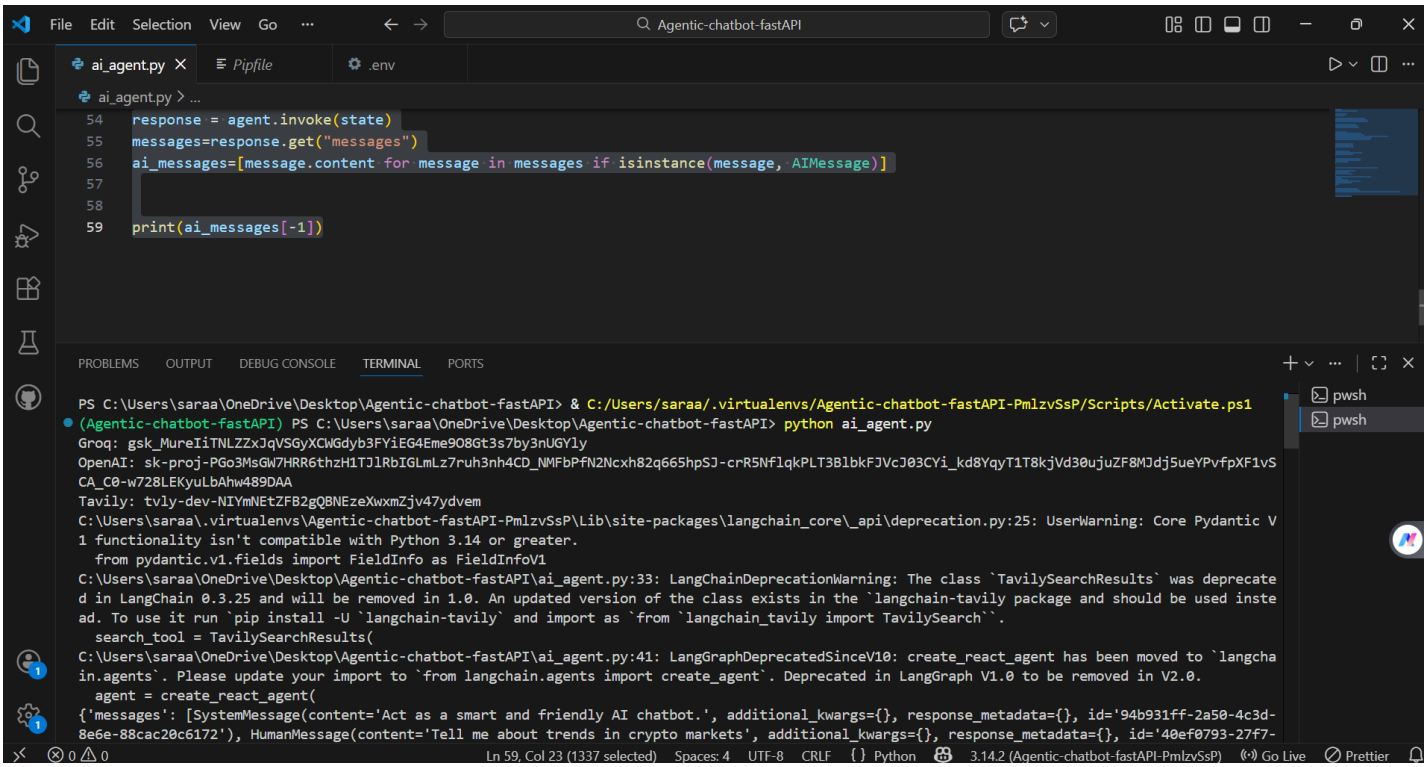
The agent is now **fully functional**:

- It receives a user query.
- Uses the Groq model to reason.
- Calls the Tavily search tool to fetch current information.
- Combines the results and returns a coherent AI-generated response.

This demonstrates a **working agentic chatbot pipeline** that integrates LLM reasoning with external tools for live data retrieval.

Summary of Achievements

- Environment and dependencies fully set up.
- API keys managed securely.
- Multi-LLM integration done.
- Tool integration implemented for dynamic information retrieval.
- Agent created with ReAct framework.
- Conversation flow and state handling implemented.
- Agent successfully invoked and producing meaningful outputs



Phase 2: Setup backend

Phase Description

In Phase 2, we developed the **backend of the Agentic Chatbot system** using **FastAPI**.
This phase focuses on building an API that receives user requests, processes them using AI models, and returns intelligent responses.

The backend integrates modern AI technologies such as:

- **LangChain**
- **LangGraph**
- **OpenAI** models
- **Groq** models
- **Tavily** search tool

The goal of this phase is to create a **dynamic AI API service** that can handle different models, providers, and optional search functionality.

What We Implemented in Backend

During this phase, we implemented the following components:

API Endpoint (/chat)

- Created a POST endpoint using FastAPI
- Accepts JSON input from frontend
- Returns AI-generated response

Request Validation (Pydantic Schema)

We created a schema to validate user input:

- model name
- model provider
- system prompt
- user messages
- search permission

This ensures **clean, structured, and safe data processing**.

Dynamic Model Selection

The backend dynamically selects the LLM based on user input:

- OpenAI → GPT model
- Groq → LLaMA model

This makes the system **flexible and scalable**.

AI Agent Creation (LangGraph)

We used **LangGraph ReAct Agent** to:

- Understand user query
- Decide whether to use search
- Generate intelligent responses

Search Tool Integration (Tavily)

If allow_search = true:

- The agent uses Tavily tool
- Retrieves real-time web information
- Improves response accuracy

Environment Variables Security

We used .env file to store:

- API keys
- Secrets

This keeps the system **secure and production-ready**.

What is Happening Internally (Flow)

Here is what happens when a user sends a request:

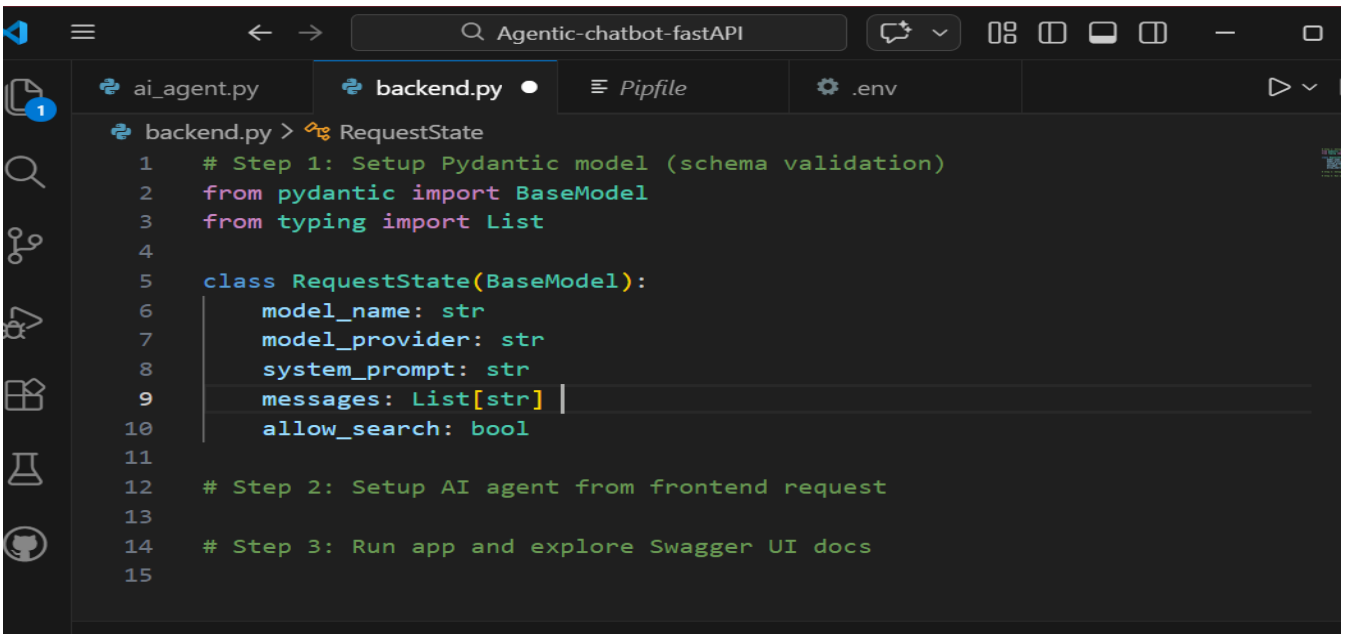
1. User sends request to /chat
2. FastAPI validates data using schema
3. System checks if model is allowed
4. Selected LLM (Groq/OpenAI) is loaded
5. Search tool is added (if enabled)
6. LangGraph agent processes the query
7. Agent generates final response
8. Backend returns response to user

Summary of Phase 2

In this phase, we successfully built a **fully functional AI backend system** that:

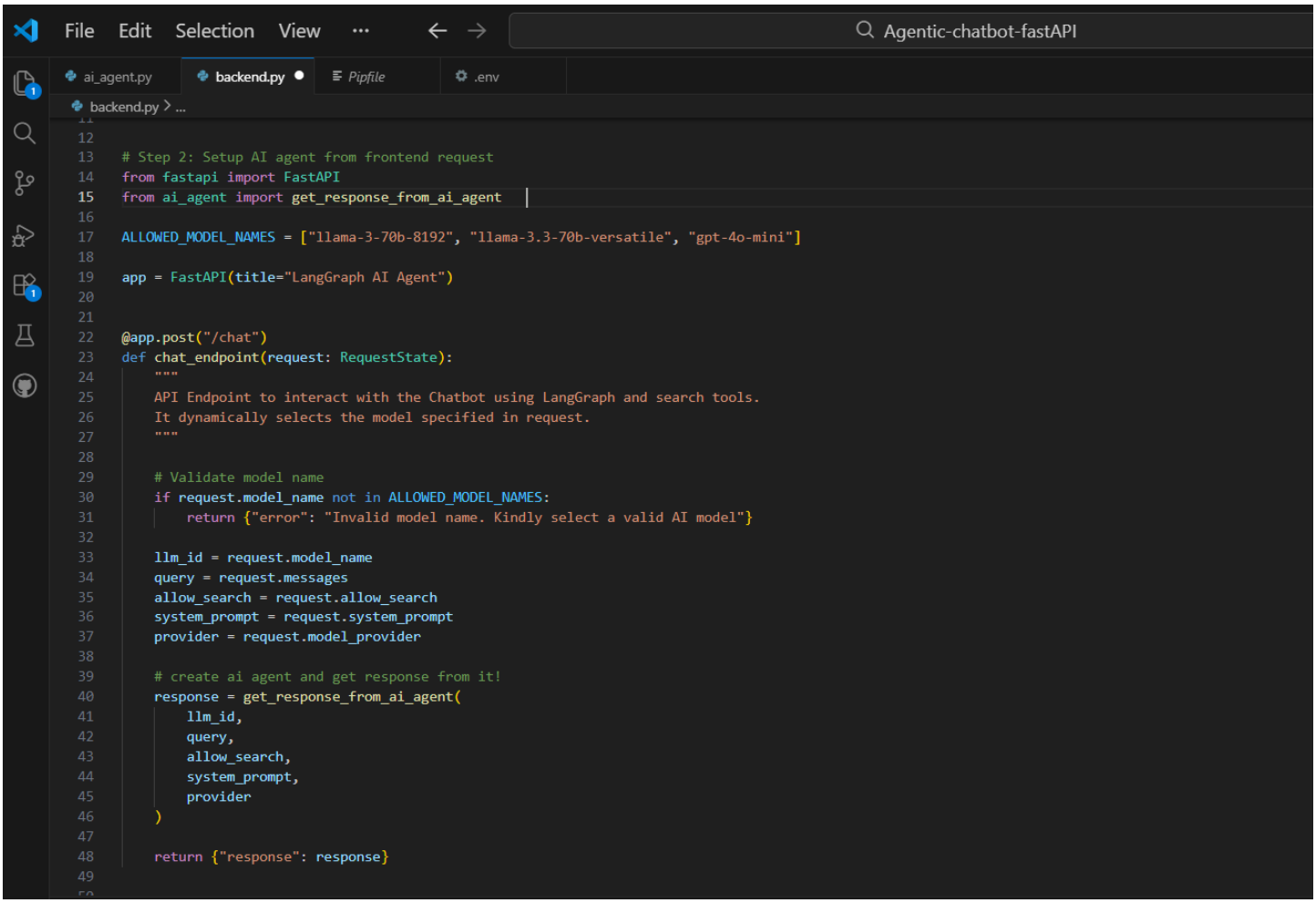
✓ Accepts user queries
✓ Selects AI model dynamically
✓ Uses intelligent agent reasoning
✓ Integrates real-time search
✓ Returns accurate AI responses

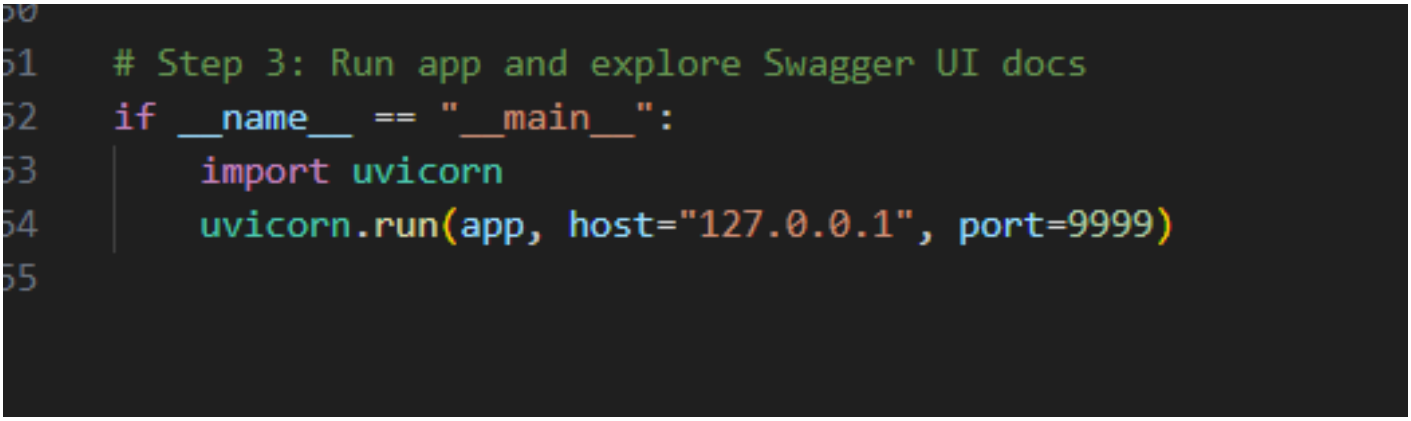
This backend acts as the **core brain of the chatbot system**.



```
(Agentic-chatbot-fastAPI) PS C:\Users\saraa\OneDrive\Desktop\Agentic-chatbot-fastAPI>
pipenv install pydantic
Loading .env environment variables...
Courtesy Notice:
Pipenv found itself running within a virtual environment,  so
it will automatically use that environment, instead of
creating its own for any project. You can set
PIPENV_IGNORE_VIRTUALENVS=1 to force pipenv to ignore that
environment and create  its own instead.
You can set PIPENV_VERBOSITY=-1 to suppress this warning.
PIPENV_IGNORE_VIRTUALENVS=1 to force pipenv to ignore that
environment and create  its own instead.
You can set PIPENV_VERBOSITY=-1 to suppress this warning.
environment and create  its own instead.
You can set PIPENV_VERBOSITY=-1 to suppress this warning.
You can set PIPENV_VERBOSITY=-1 to suppress this warning.
To activate this project's virtualenv, run pipenv shell.
Alternatively, run a command inside the virtualenv with pipenv
run.
Installing pydantic...
```

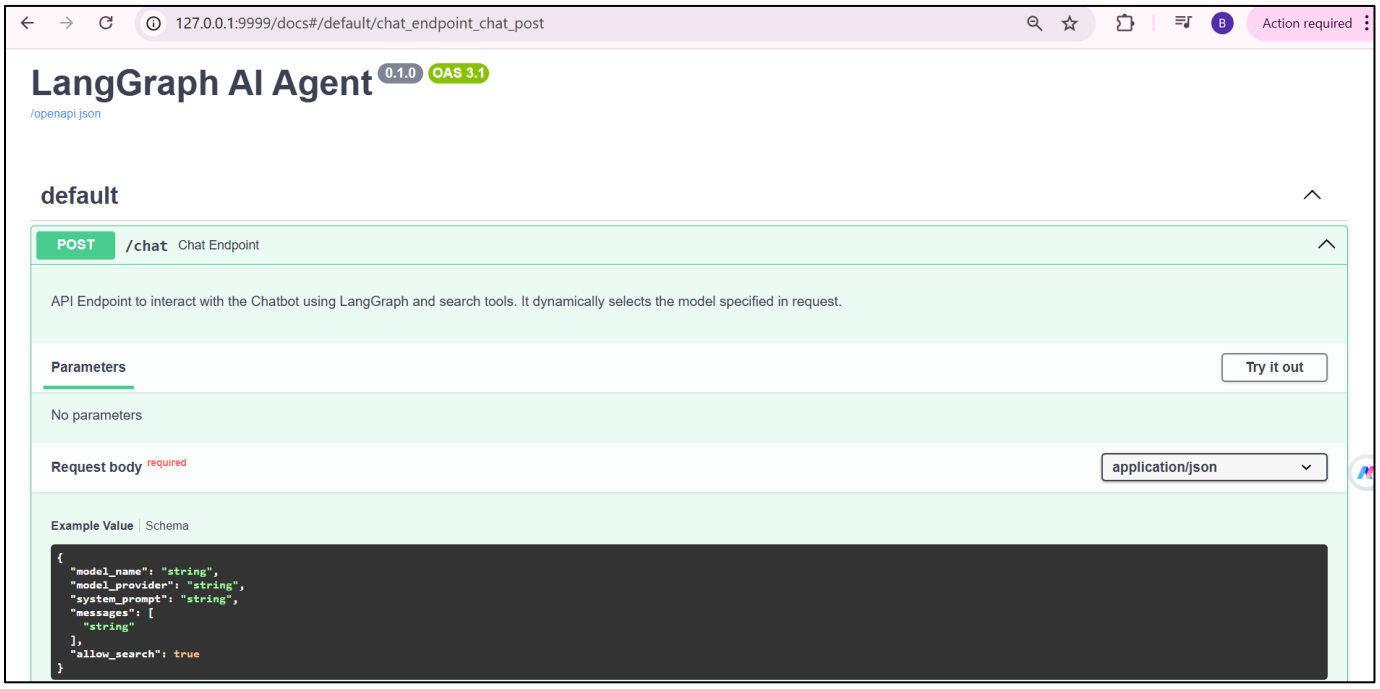
```
Installing pydantic...
Installation Succeeded
To activate this project's virtualenv, run pipenv shell.
Alternatively, run a command inside the virtualenv with pipenv
run.
Installing dependencies from Pipfile.lock (cdcc2b)...
All dependencies are now up-to-date!
Upgrading pydantic in  dependencies.
Building requirements...
Resolving dependencies...
Success!
Building requirements...
Resolving dependencies...
Success!
To activate this project's virtualenv, run pipenv shell.
Alternatively, run a command inside the virtualenv with pipenv run.
Installing dependencies from Pipfile.lock (a36237)...
All dependencies are now up-to-date!
Installing dependencies from Pipfile.lock (a36237)...
(Agentic-chatbot-fastAPI) PS C:\Users\saraa\OneDrive\Desktop\Agentic-chatbot-fastAPI>
```

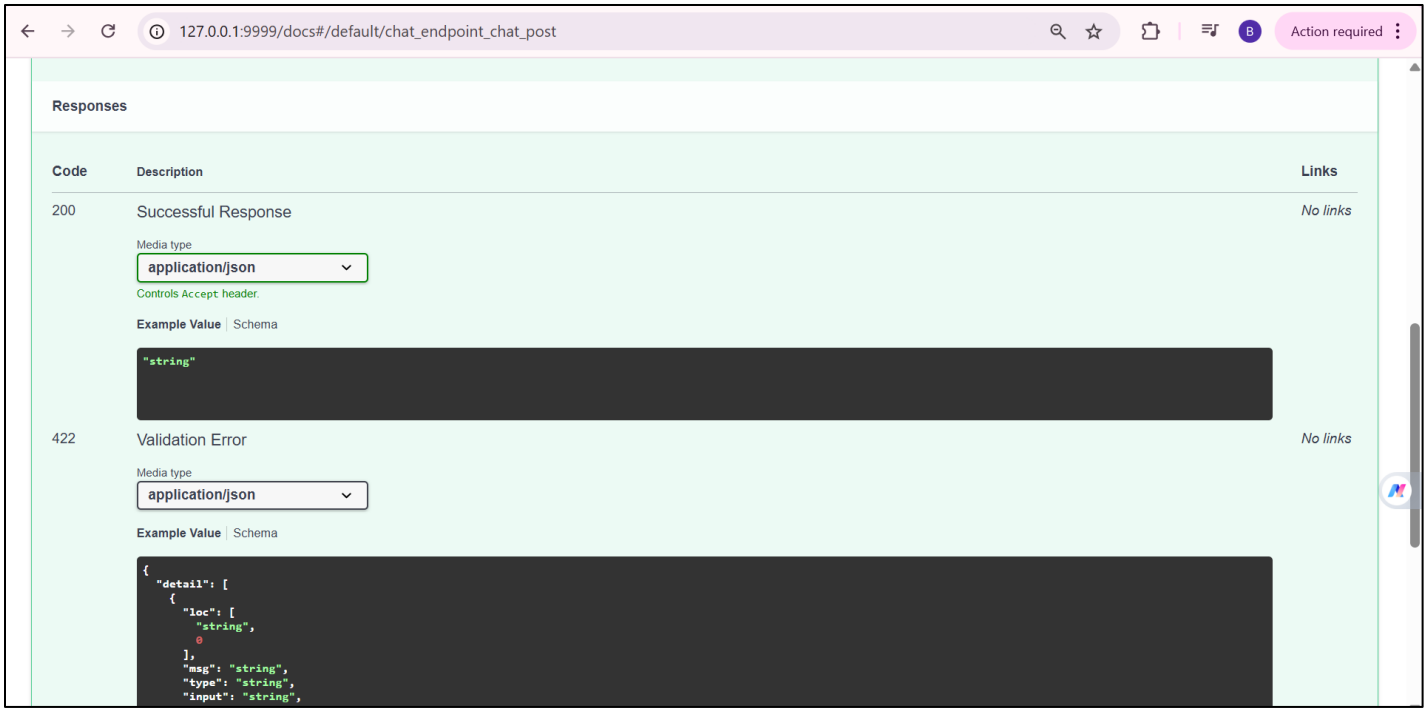




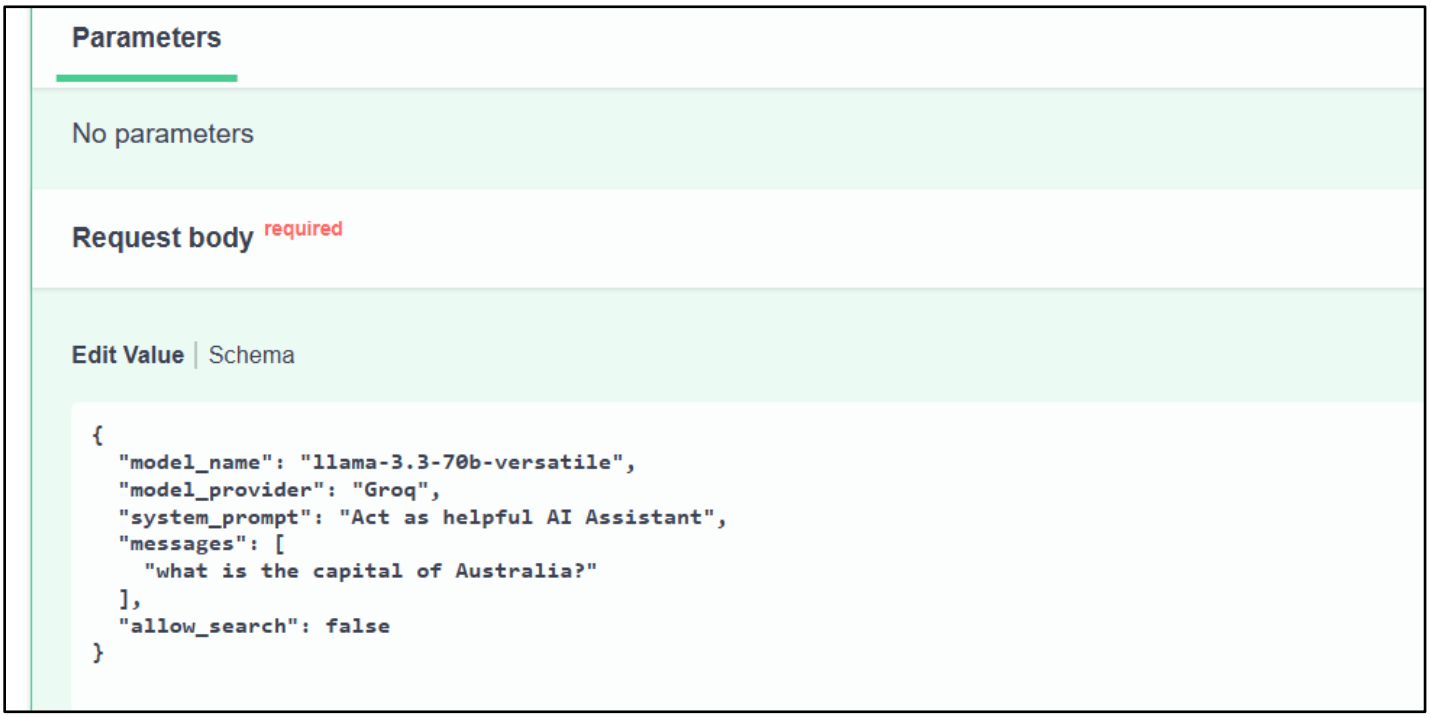
In Phase 2, we developed the backend of the Agentic Chatbot using FastAPI, integrating LangGraph agents with OpenAI and Groq LLMs, along with the Tavily search tool for dynamic responses. The backend includes a /chat POST API endpoint, which validates user input using a Pydantic schema to ensure structured and correct requests. The system was run locally on 127.0.0.1:9999, and the Swagger UI confirmed that the API, validation, and AI agent workflow were functioning correctly, making the backend ready for deployment.

```
54    uvicorn.run(app, host="127.0.0.1", port=9999)

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(Agentic-chatbot-fastAPI) PS C:\Users\saraa\OneDrive\Desktop\Agentic-chatbot-fastAPI>pipenv shell
C:\Users\saraa\OneDrive\Desktop\Agentic-chatbot-fastAPI\ai_agent.py:35: LangChainDeprecationWarning: The class `TavilySearchResults` was deprecated in LangChain
lass exists in the `langchain-tavily package and should be used instead. To use it run `pip install -U `langchain-tavily` and import as `from `langchain_tavily
  search_tool = TavilySearchResults(
INFO:     Started server process [29724]
INFO:     Waiting for application startup.
 install -U `langchain-tavily` and import as `from `langchain_tavily import TavilySearch``.
  search_tool = TavilySearchResults(
INFO:     Started server process [29724]
INFO:     Waiting for application startup.
  search_tool = TavilySearchResults(
INFO:     Started server process [29724]
INFO:     Waiting for application startup.
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:9999 (Press CTRL+C to quit)
INFO:     127.0.0.1:57229 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:57229 - "GET /openapi.json HTTP/1.1" 200 OK
```





Test your backend:



Curl

```
curl -X 'POST' \
  'http://127.0.0.1:9999/chat' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "model_name": "llama-3.3-70b-versatile",
  "model_provider": "Groq",
  "system_prompt": "Act as helpful AI Assistant",
  "messages": [
    "what is the capital of Australia?"
  ],
  "allow_search": false
}'
```

Request URL

```
http://127.0.0.1:9999/chat
```

Server response

Code	Details
200	Response body

```
{
  "response": "The capital of Australia is Canberra. It's located in the Australian Capital Territory (ACT) and has been the country's capital since 1908. Canberra is home to many national institutions, including the Parliament of Australia, the High Court of Australia, and several national museums and galleries."
}
```

Download

Response headers

```
content-length: 314
content-type: application/json
date: Wed,18 Feb 2026 14:50:31 GMT
server: uvicorn
```

Responses

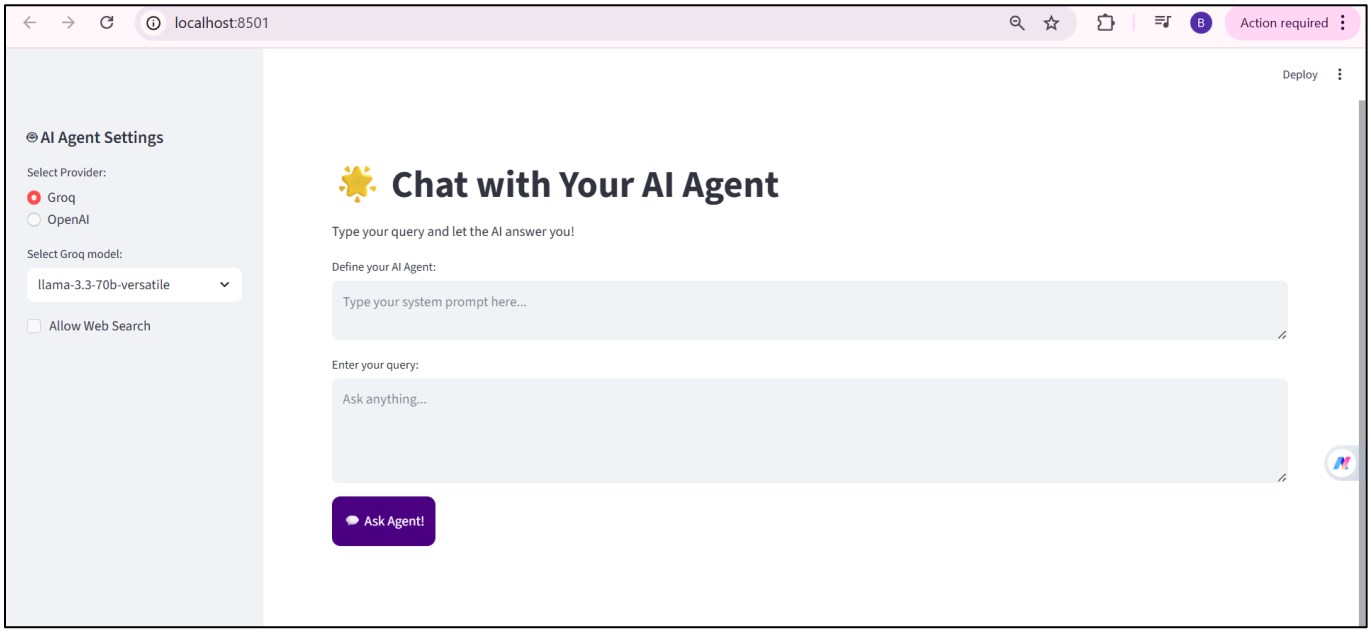
Phase 3: Frontend UI Implementation (Streamlit)

In Phase 3, we focused on creating a user-friendly frontend interface for the AI Chat Agent using Streamlit. The goal was to allow users to easily interact with the backend AI models (Groq and OpenAI) without writing any code.

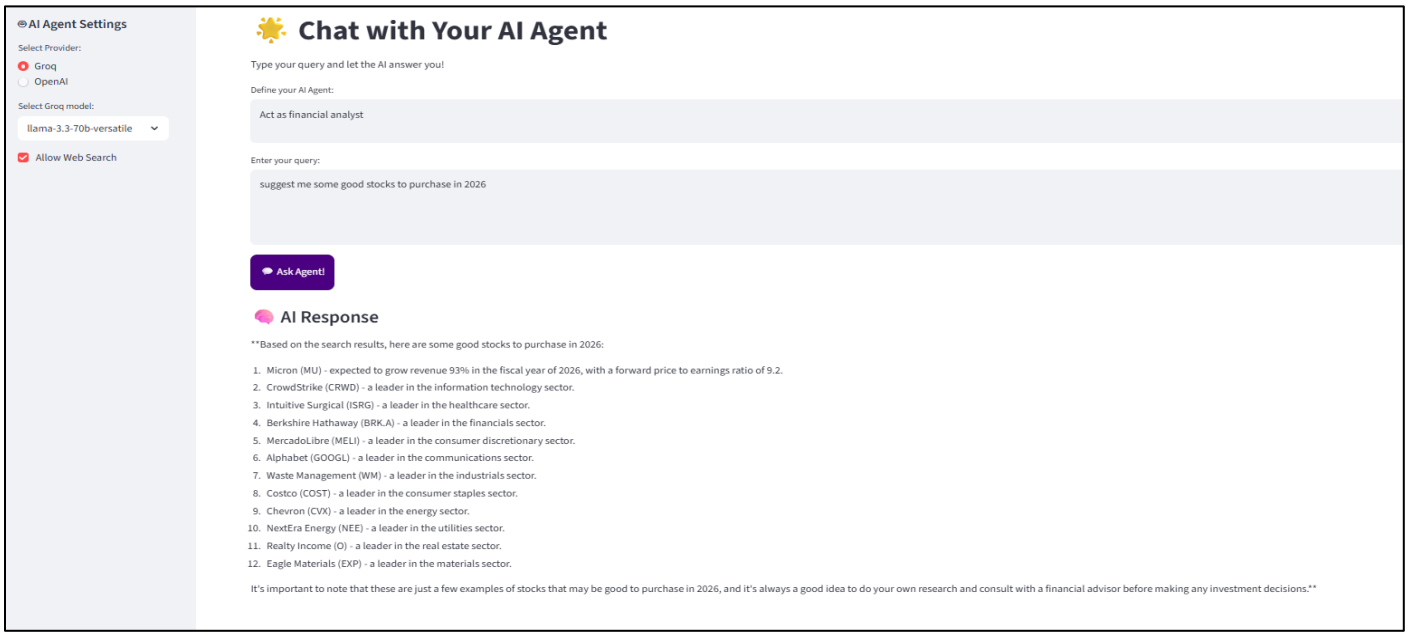
What we did:

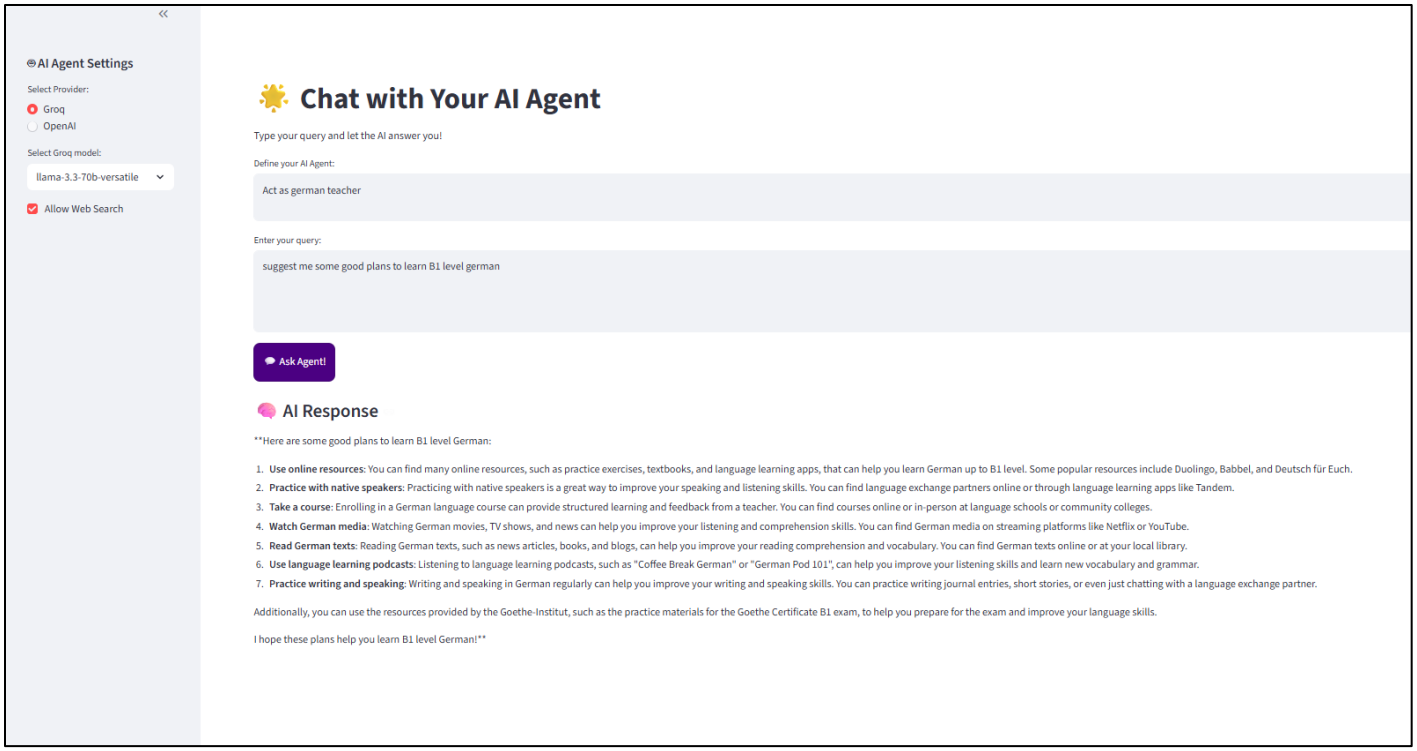
1. UI Setup and Page Config:
    - Configured the page with st.set_page_config for wide layout and a custom title.
    - Added aesthetic improvements using custom CSS for background gradients, styled buttons, and text areas to make the interface visually appealing.
2. Sidebar for Model and Settings:
    - Created a sidebar to allow users to select:
        - Provider: Groq or OpenAI
        - Model: Different models per provider (e.g., llama-3.3-70b-versatile for Groq, gpt-4o-mini for OpenAI)
        - Allow Web Search: Optional toggle to include search-based results
    - This lets users dynamically configure the AI agent before sending a query.
3. Main Page for Chat Interaction:
    - Added fields for:
        - System Prompt: To define the behavior or personality of the AI agent.
        - User Query: Where users type their questions.
    - A “Ask Agent” button sends the query to the backend.

4. Connecting to Backend:
    - The frontend sends a POST request to the FastAPI backend at http://127.0.0.1:9999/chat with the selected model, provider, system prompt, user query, and search preference.
    - Handles response:
        - If successful, displays the AI response.
        - If there's an error (invalid model or server offline), shows appropriate messages.
5. Error Handling:
    - Includes a check for backend server availability.
    - Gives user feedback if the backend is not running, preventing the app from crashing.



Final test :




AI Agent Settings
Select Provider:
Groq
OpenAI
Select Groq model:
llama-3.3-70b-versatile
Allow Web Search
Chat with Your AI Agent
Type your query and let the AI answer you!
Define your AI Agent:
Act as german teacher
Enter your query:
suggest me some good plans to learn B1 level german
Ask Agent!
AI Response
**Here are some good plans to learn B1 level German:
1. Use online resources: You can find many online resources, such as practice exercises, textbooks, and language learning apps, that can help you learn German up to B1 level. Some popular resources include Duolingo, Babbel, and Deutsch für Euch.
2. Practice with native speakers: Practicing with native speakers is a great way to improve your speaking and listening skills. You can find language exchange partners online or through language learning apps like Tandem.
3. Take a course: Enrolling in a German language course can provide structured learning and feedback from a teacher. You can find courses online or in-person at language schools or community colleges.
4. Watch German media: Watching German movies, TV shows, and news can help you improve your listening and comprehension skills. You can find German media on streaming platforms like Netflix or YouTube.
5. Read German texts: Reading German texts, such as news articles, books, and blogs, can help you improve your reading comprehension and vocabulary. You can find German texts online or at your local library.
6. Use language learning podcasts: Listening to language learning podcasts, such as "Coffee Break German" or "German Pod 101", can help you improve your listening skills and learn new vocabulary and grammar.
7. Practice writing and speaking: Writing and speaking in German regularly can help you improve your writing and speaking skills. You can practice writing journal entries, short stories, or even just chatting with a language exchange partner.
Additionally, you can use the resources provided by the Goethe-Institut, such as the practice materials for the Goethe Certificate B1 exam, to help you prepare for the exam and improve your language skills.
I hope these plans help you learn B1 level German!**

****